

НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ УКРАЇНИ
«КИЇВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ
імені ІГОРЯ СІКОРСЬКОГО»

факультет інформатики та обчислювальної техніки
(повна назва інституту/факультету)

кафедра автоматика та управління в технічних системах
(повна назва кафедри)

Курсова робота

з дисципліни «Програмування-3»

на тему: Система проведення аукціонів

Виконав : студент 1 курсу, групи ІА-42
(шифр групи)

Осадчий Дмитро Олександрович
(прізвище, ім'я, по батькові) _____ (підпис)

Науковий керівник _____ (підпис)
(посада, науковий ступінь, вчене звання, прізвище та ініціали)

Члени комісії _____ (підпис)
(посада, науковий ступінь, вчене звання, прізвище та ініціали)

_____ (підпис)
(посада, науковий ступінь, вчене звання, прізвище та ініціали)

Засвідчую, що у цій магістерській дисертації немає запозичень з праць інших авторів без відповідних посилань.

Студент
Осадчий Д. О. _____ (підпис)

ЗМІСТ

ВСТУП	3
1 ВИМОГИ ДО СИСТЕМИ	4
1.1 Функціональні вимоги до системи	4
1.2 Нефункціональні вимоги до системи	5
2 СЦЕНАРІЇ ВИКОРИСТАННЯ СИСТЕМИ	5
2.1 Діаграма прецедентів	5
2.2 Опис сценаріїв використання системи	6
3 АРХІТЕКТУРА СИСТЕМИ	10
4 РЕАЛІЗАЦІЯ КОМПОНЕНТІВ СИСТЕМИ	12
4.1 Загальна структура проекту	12
4.2 Компоненти рівня доступу до даних	13
4.3 Компоненти рівня бізнес-логіки	23
4.4 Компоненти рівня	25
ВИСНОВКИ	29
ПЕРЕЛІК ВИКОРИСТАНИХ ДЖЕРЕЛ	30

ВСТУП

Метою цієї курсової роботи є створення системи для проведення аукціонів, яка дозволить користувачам легко створювати лоти, робити ставки та управляти торгами. Система повинна мати зручний інтерфейс для перегляду активних лотів, пошуку товарів за ключовими словами, а також функції для запуску та зупинки аукціонів для їхніх власників. Крім того, важливо, щоб система забезпечувала безпеку даних, коректну обробку ставок і автоматичне оновлення інформації про лоти. Ще одним важливим аспектом є можливість створення унікальних посилань для швидкого доступу до конкретних лотів, що спростить їх поширення. В результаті, ми плануємо створити працюючий прототип, який продемонструє основні сценарії використання: від додавання нового лоту до завершення торгів з визначенням переможця. Ця робота стане основою для подальшого вдосконалення системи, наприклад, шляхом додавання функцій для платежів, рейтингів користувачів або розширеного пошуку.

Сучасний світ стрімко розвивається в цифровому напрямку, і електронна комерція не є винятком. Онлайн-аукціони стають все більш популярними серед покупців і продавців по всьому світу, пропонуючи зручний і прозорий спосіб торгівлі різноманітними товарами — від унікальних колекційних предметів до нерухомості та цифрових активів. Проте багато існуючих платформ мають суттєві недоліки: складний інтерфейс, обмежений функціонал або завищені комісії, що значно знижує їх привабливість для користувачів. Тому розробка власної системи проведення аукціонів є надзвичайно актуальним завданням, яке може запропонувати користувачам сучасне, зручне та безпечне рішення для онлайн-торгів.

У межах цієї курсової роботи ми створили повноцінний багатосторінковий веб-сайт для проведення аукціонів, який має всі необхідні функції для зручної роботи користувачів. Для розробки фронтенду ми використали стандартні веб-технології HTML та CSS, що дозволило створити зручний і інтуїтивно зрозумілий інтерфейс. Бекенд-система була реалізована на мові програмування Python, що забезпечило надійну роботу серверної частини.

Особливу увагу було приділено організації зберігання даних. Для цього ми обрали реляційну базу даних MySQL, яка чудово справляється з усією необхідною інформацією: даними про зареєстрованих користувачів, детальними описами лотів, історією ставок та змінами цін, а також інформацією про активність аукціонів. Наша система забезпечує повний цикл роботи з даними - від їх коректного зберігання після перезавантаження до надійного видалення, коли це потрібно.

Важливою частиною проекту стало впровадження механізму реєстрації та автентифікації користувачів. Наша система гарантує надійний вхід і вихід з акаунту, а також має розвинену систему валідації даних, яка ретельно перевіряє правильність заповнення полів під час реєстрації нових користувачів. Це допомагає уникнути помилок і забезпечує цілісність даних у системі.

Розроблений веб-додаток надає користувачам зручний інструмент для розміщення лотів, дозволяє брати участь у торгах у реальному часі та гарантує безпеку угод. В результаті ми отримали повноцінну функціонуючу систему, яка може використовуватися для організації різноманітних онлайн-аукціонів. Проект має великий потенціал для подальшого розвитку, включаючи впровадження платіжних систем, розширення аналітичних можливостей або інтеграцію з іншими сервісами, що відкриває нові перспективи для його практичного використання.

1 ВИМОГИ ДО СИСТЕМИ

1.1 Функціональні вимоги до системи:

Система має відповідати наступним функціональним вимогам:

- Незареєстрований користувач повинен мати можливість переглядати лоти, інформацію про лоти, та завершені лоти;
- Незареєстрований користувач повинен мати можливість здійснювати пошук лотів по словам у назві;

- Незареєстрований користувач повинен мати можливість переглядати ставки інших користувачів;
- Зареєстрований користувач повинен мати усі можливості, що є у незареєстрованого користувача, а також він повинен мати можливість створювати свої лоти, редагувати свої лоти, та видаляти або завершувати торги за свої лоти, приймати участь у торгах за лоти інших користувачів.

1.2 Нефункціональні вимоги до системи:

- Система повинна мати відкриту архітектуру;
- Система повинна мати веб-інтерфейс;
- Інтерфейс користувача має бути зручним та інтуїтивно-зрозумілим;
- Система повинна бути крос-платформною;

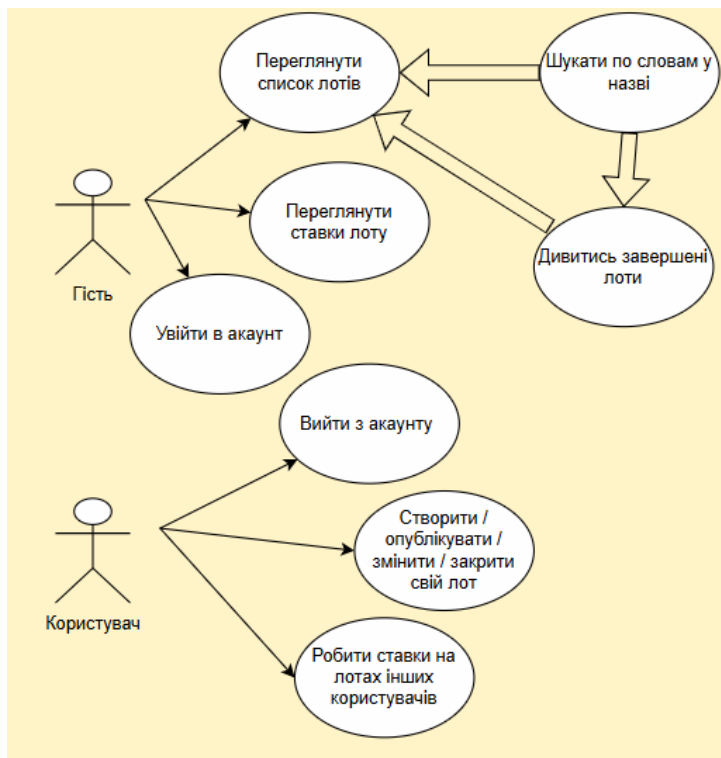
2 СЦЕНАРІЇ ВИКОРИСТАННЯ СИСТЕМИ

2.1 Діаграма прецедентів

Діаграма прецедентів системи представлена на мал. 2.1.

Акторами є користувачі системи: незареєстрований (гість) та зареєстрований (користувач).

Зареєстрованому користувачу доступна уся функціональність, що і незареєстрованому, а також можливість створювати / змінювати / закривати / видаляти свої лоти, приймати участь в лотах інших користувачів. Детально усі сценарії використання описані у наступному підрозділі.



Малюнок 2.1 – Діаграма прецедентів

2.2 Опис сценаріїв використання системи

Детальні описи сценаріїв використання наведено у таблицях 2.1 –

Таблиця 2.1 – Сценарій використання «Пошук по назві лоту»

Назва	Пошук по назві лоту
ID	1
Опис	Користувач, використовуючи поле для пошуку, шукає лот за його назвою
Актори	Гість, Користувач
Вигоди компанії	Якщо користувачі не можуть шукати потрібні їм лоти, то вони навряд чи продовжать користуватись сервісом
Частота користування	Постійно
Тригери	Користувач вводить пошуковий запит у полі для пошуку
Передумови	Пошукове поле доступне у будь-якому вікні
Постумови	Користувач потрапляє на лот який шукав

Основний розвиток	Користувач вводить запит у пошукову строку, натискає на кнопку пошуку
Альтернативні розвитку	-
Виняткові ситуації	-

Таблиця 2.2 – Сценарій використання «Перегляд лотів»

Назва	Перегляд лотів
ID	2
Опис	Користувач обирає лот, який хоче подивитись або купити
Актори	Користувач
Вигоди компанії	Стримінговий сервіс без лотів не буде користуватися попитом
Частота користування	Постійно
Тригери	Користувач натискає на кнопку “Відкрити лот ” біля лоту
Передумови	У вікні є лоти, які можна переглянути
Постумови	Лот продається
Основний розвиток	Користувач натискає кнопку «Відкрити лот» Якщо користувача лот цікавить, то він приймає в ньому участь Лот продається
Альтернативні розвитку	-
Виняткові ситуації	Відмова від участь в продажу лоту Закриття лоту

В таблиці 2.3 представлений сценарій використання «Перегляд головної сторінки із лотами»

Таблиця 2.3 – Сценарій використання «Перегляд головної сторінки із лотами та рекомендаціями»

Назва	Перегляд головної сторінки із лотами та рекомендаціями
ID	3
Опис	Користувач переглядає головну сторінку із лотами та рекомендаціями

Актори	Користувач
Вигоди компанії	Головна сторінка важлива для зацікавлення нових користувачів, та актуальна інформація буде утримувати старих
Частота користування	Постійно
Тригери	Користувач переходить на сайт/приймає участь в лотах
Передумови	Немає
Постумови	Користувач потрапляє на головну сторінку
Основний розвиток	Користувач запускає додаток/переходить на сайт
Альтернативні розвитку	-
Виняткові ситуації	Сервіс недоступний у країні користувача, тоді показується повідомлення про помилку

В таблиці 2.4 представлений сценарій використання «Перегляд закритих лотів»

Таблиця 2.4 – Сценарій використання «Перегляд закритих лотів»

Назва	Перегляд закритих лотів
ID	4
Опис	Користувач переглядає сторінку з закритими лотами
Актори	Користувач
Вигоди компанії	Сторінка з закритими лотами важлива для зберігання інформації та інформування нових користувачів з приводу закритих лотів
Частота користування	Постійно
Тригери	Користувач переходить на сайт/передивляється закриті лоти щоб ознайомитись які позиції бувають
Передумови	Зорієнтуватись за якими приблизно цінами закриваються лоти
Постумови	Більше мотивації приймати участь у «битві» за лоти
Основний розвиток	Користувач переходить на головну сторінку та приймає участь у «битві» за лоти
Альтернативні розвитку	-
Виняткові ситуації	Сервіс недоступний у країні користувача, тоді показується повідомлення про помилку

В таблиці 2.5 представлений сценарій використання «Додавання та публікація нових лотів»

Таблиця 2.5 – Сценарій використання «Додавання та публікація нових лотів»

Назва	Додавання та публікація нових лотів
ID	5
Опис	Користувач публікує свої лоти
Актори	Користувач
Вигоди компанії	Йде оборот товару на сайті, це закликає нових зацікавлених в купівлі будь чого на сайті
Частота користування	Часто
Тригери	Користувач натискає на відповідну кнопку
Передумови	Користувач знаходиться на сторінці із лотами, обирає якийсь із них
Постумови	Публікація своїх лотів
Основний розвиток	Користувач натискає на відповідну кнопку біля публікації лоту та публікує його
Альтернативні розвитку	Користувач натискає на відповідну кнопку для створення лоту . Відкривається віконце із формою заповнення інформації про лот, та користувач створює та залишає не опублікованим
Виняткові ситуації	-

3 АРХІТЕКТУРА СИСТЕМИ

Загальна архітектура системи наведена на рис. 3.1

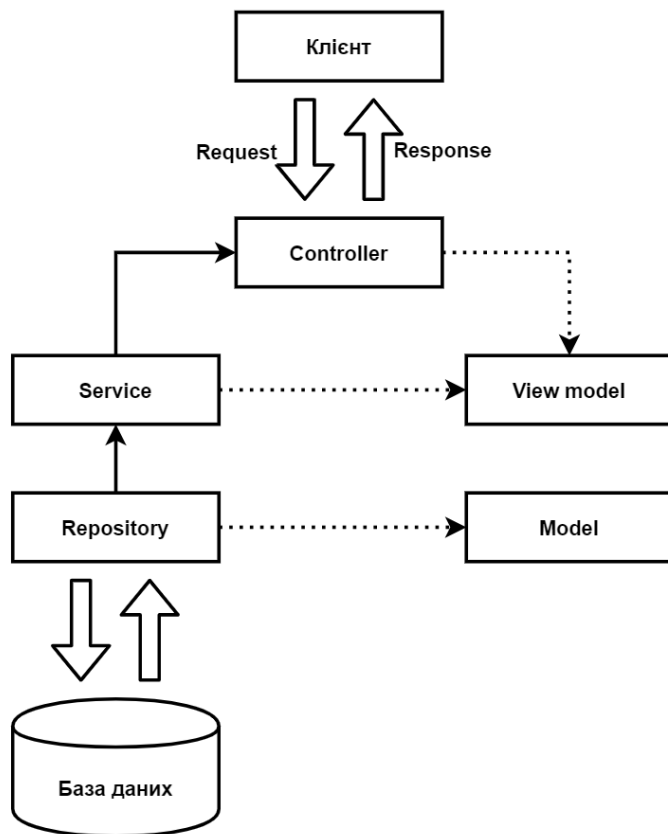


Рисунок 3.1 – Загальна архітектура системи

Система складається з таких компонентів:

1. Клієнтська частина
2. Серверна частина
3. База даних
4. Додаткові компоненти

Система проведення аукціонів включає клієнтську частину, яка надає зручний інтерфейс для користувачів, дозволяючи їм переглядати лоти, робити ставки та керувати своїми аукціонами через веб- або мобільний додаток. Серверна частина обробляє всі запити від клієнта, включаючи створення лотів, обробку ставок та управління аукціонами, використовуючи API, контролери, сервіси та репозиторії для забезпечення коректної роботи бізнес-логіки. База даних зберігає всю критичну інформацію, таку як дані лотів, ставок та користувачів, забезпечуючи швидкий доступ та надійне зберігання. Додаткові компоненти, такі як сервіс сповіщень, кешування або файлове сховище, можуть бути інтегровані для покращення продуктивності та

функціональності системи, надаючи користувачам більш комфортний досвід взаємодії.

Серверна частина включає наступні компоненти:

1. Контролери
2. Моделі
3. Сервіси
4. Репозиторії
5. Маршрутизація

Серверна складова системи організована таким чином, щоб продуктивно опрацьовувати всі вхідні запити від клієнтської частини. Контролери відповідають за приймання HTTP-запитів, їх початкову обробку та маршрутизацію до відповідних сервісів, виступаючи посередниками між клієнтом і бізнес-логікою. Моделі визначають структуру даних і бізнес-правила, описуючи основні сутності системи, такі як лоти, ставки та юзери. Сервіси містять головну логіку програми – вони виконують складні обчислення, перевірки та операції, пов'язані з аукціонами, забезпечуючи правильність всіх процесів. Репозиторії відповідають за взаємодію з базою даних, виконуючи операції збереження, отримання, оновлення та видалення даних, абстрагуючи роботу зі сховищем від основної логіки. Маршрутизація керує тим, які контролери обробляють конкретні запити, визначаючи шляхи API та забезпечуючи коректну передачу даних між клієнтом і сервером. Усі ці компоненти тісно взаємодіють, забезпечуючи стабільну і безпечну роботу системи в цілому.

4.1 Загальна структура проекту

Загальна структура проекту представлена на рисунках 4.1.1 та 4.1.2

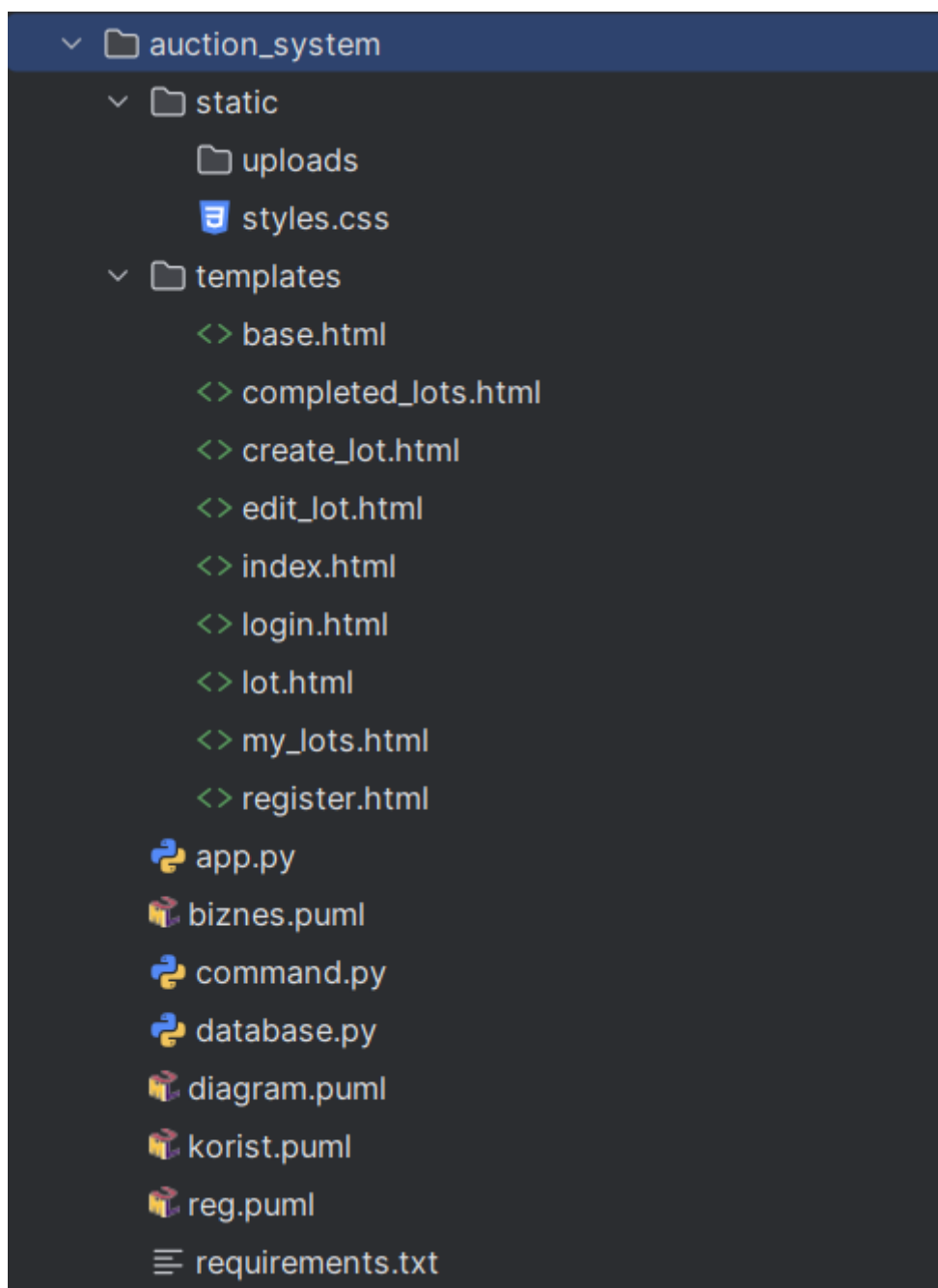


Рисунок 4.1.1 – Загальна структура проекту

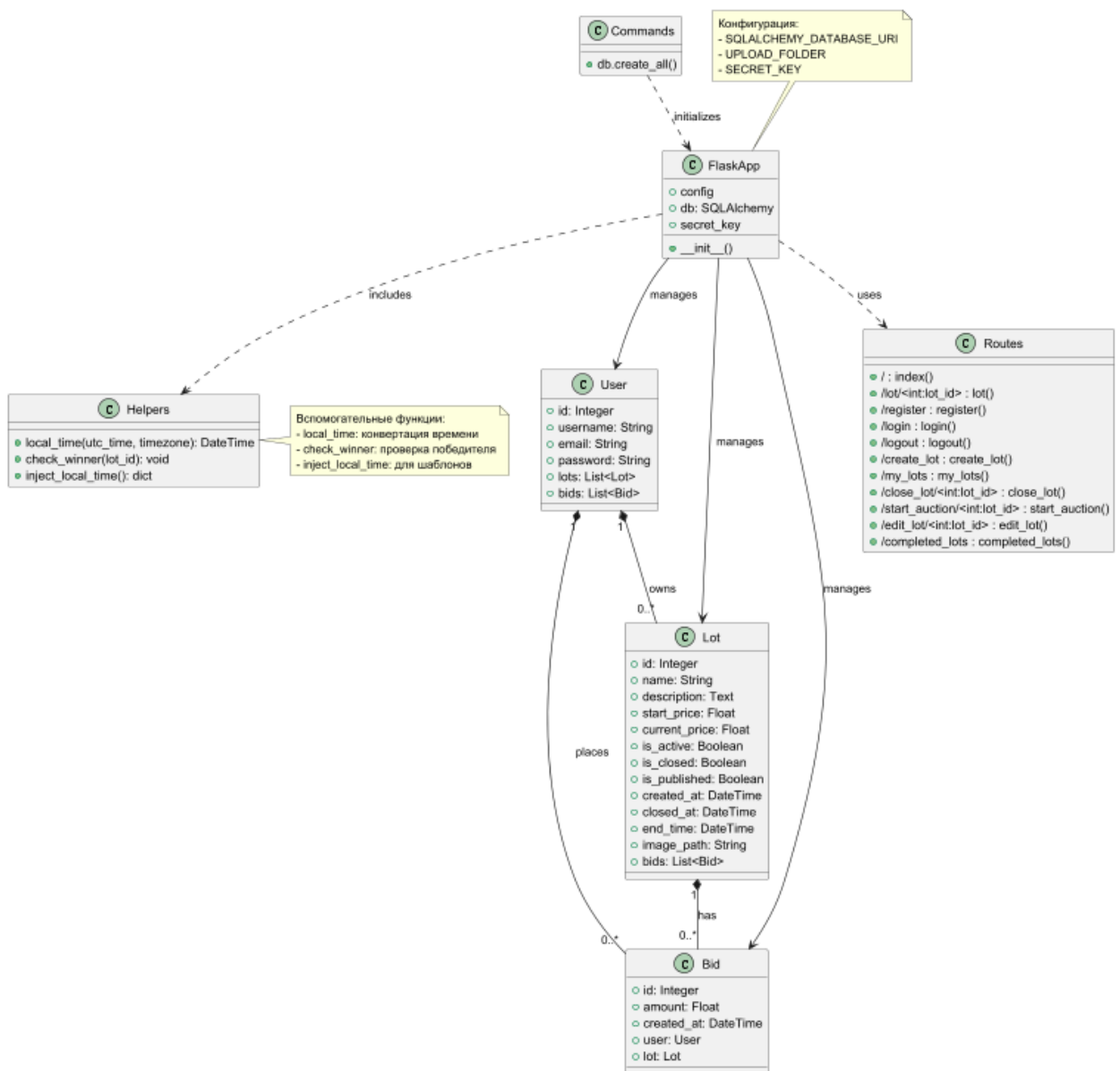


Рисунок 4.1.2 – Загальна структура проекту

Проект включає веб-ресурси, бібліотеки і вихідний код, який можна поділити на компоненти рівня доступу до даних, компоненти бізнес-логіки, компоненти авторизації(Security) та веб-компоненти.

4.2 Компоненти рівня доступу до даних

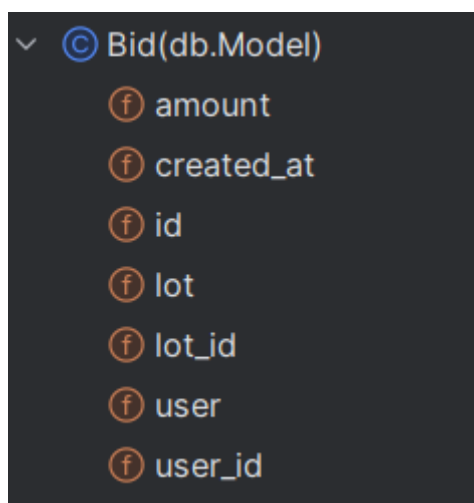
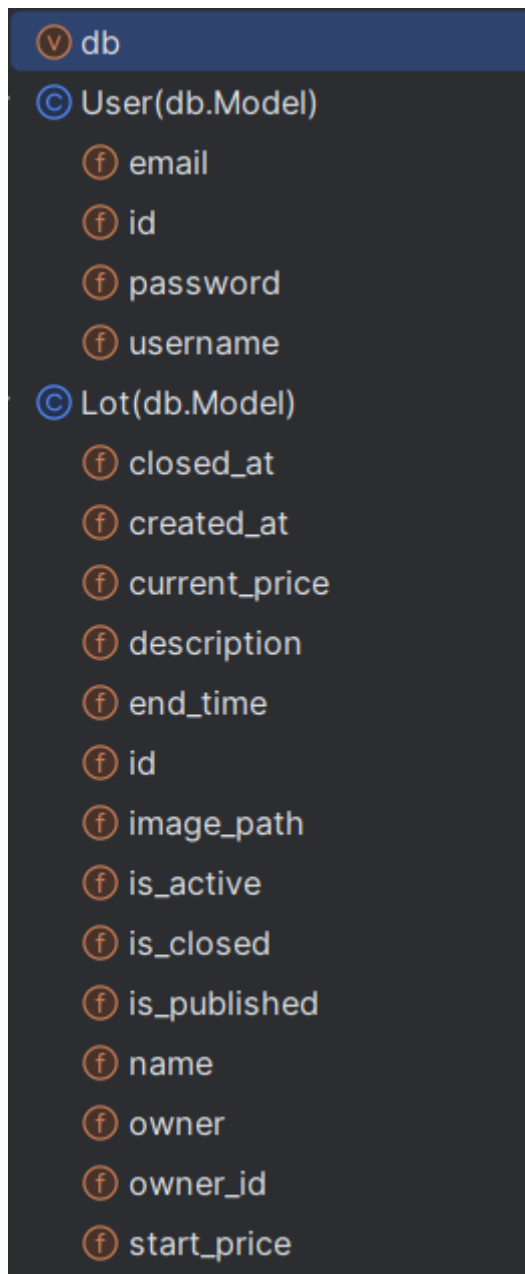


Рисунок 4.2 – Основні сутності та інтерфейси рівня доступу до даних

Детальний опис сутностей та інтерфейсів:

Database.py:

1. Клас User (Користувач)

- **Призначення:** Модель користувача системи
- **Атрибути:**
 - id: унікальний ідентифікатор (первинний ключ)
 - username: унікальне ім'я користувача (обов'язкове)
 - email: унікальна електронна пошта (обов'язкова)
 - password: пароль (обов'язковий)
- **Зв'язки:**
 - Має відношення "один-до-багатьох" з моделлю Lot (користувач може створити багато лотів)
 - Має відношення "один-до-багатьох" з моделлю Bid (користувач може робити багато ставок)
- **Особливості:**
 - Усі поля обов'язкові для заповнення
 - Поля username та email мають унікальність на рівні бази даних

2. Клас Lot (Аукціонний лот)

- **Призначення:** Модель аукціонного лоту
- **Атрибути:**
 - id: унікальний ідентифікатор
 - name: назва лоту (обов'язкова)
 - description: опис лоту (обов'язковий)
 - start_price: початкова ціна (обов'язкова)
 - current_price: поточна ціна (обов'язкова)
 - is_active: чи активний лот (за замовчуванням True)

- is_closed: чи закритий лот (за замовчуванням False)
- is_published: чи опублікований лот (за замовчуванням False)
- created_at: дата створення (автоматично встановлюється)
- closed_at: дата закриття
- end_time: час завершення аукціону
- image_path: шлях до зображення лоту
- **Зв'язки:**
 - Належить користувачу (власнику) через owner_id
 - Має багато ставок (Bid)
- **Особливості:**
 - Автоматичне встановлення часу створення
 - Може знаходитись у різних станах (активний/закритий/опублікований)

3. Клас Bid (Ставка)

- **Призначення:** Модель ставки на аукціоні
- **Атрибути:**
 - id: унікальний ідентифікатор
 - amount: сума ставки (обов'язкова)
 - created_at: дата створення (автоматично встановлюється)
- **Зв'язки:**
 - Належить користувачу (хто зробив ставку)
 - Належить лоту (на який зроблено ставку)
- **Особливості:**
 - Автоматичне встановлення часу ставки
 - Обов'язкові зв'язки з користувачем і лотом

4. Функціональність системи

Аукціонна логіка (AuctionLogic):

- check_winner(lot_id): автоматично перевіряє та визначає переможця аукціону через 2 хвилини після останньої ставки

- `close_lot(lot_id)`: закриває лот вручну
- `start_auction(lot_id)`: публікує лот для участі в аукціоні

Робота з часом (TimeUtils):

- `local_time(utc_time, timezone)`: конвертує UTC час у локальний
- `inject_local_time()`: ін'єкція функції форматування часу у шаблони

Контролери:

- **LotController**: управління лотами (створення, редагування, перегляд)
- **BidController**: робота зі ставками (розміщення нових ставок)
- **AuthController**: автентифікація (реєстрація, вхід, вихід)

Зв'язки між компонентами:

- Користувач може мати багато лотів і ставок
- Лот містить багато ставок і належить одному користувачу
- Ставка належить одному користувачу і одному лоту
- Бізнес-логіка взаємодіє з моделями для оновлення статусів та перевірок

Валідація:

- Усі обов'язкові поля перевіряються на рівні бази даних
- Додаткові перевірки в бізнес-логіці (наприклад, що ставка вища за поточну ціну)

App.py:

1. Конфігурація додатку

- Ініціалізація Flask-додатку з SQLite базою даних
- Налаштування шляху для зберігання завантажених зображень
- Встановлення секретного ключа для сесій
- Ініціалізація SQLAlchemy ORM

2. Допоміжні функції

- `local_time()`: конвертує UTC час у локальний часовий пояс (за замовчуванням Київ)
- `inject_local_time()`: робить функцію `local_time` доступною у всіх шаблонах
- `check_winner()`: автоматично визначає переможця аукціону через 2 хвилини після останньої ставки

3. Маршрути (Routes)

Аутентифікація:

- `/register`: Реєстрація нового користувача з валідацією унікальності імені
- `/login`: Вхід в систему з перевіркою облікових даних
- `/logout`: Вихід з системи з очищенням сесії

Робота з лотами:

- `/`: Головна сторінка з пошуком активних лотів
- `/create_lot`: Створення нового лоту з можливістю завантаження зображення
- `/my_lots`: Перегляд власних лотів (тільки для авторизованих)
- `/lot/<int:lot_id>`: Детальна інформація про лот та можливість робити ставки
- `/edit_lot/<int:lot_id>`: Редагування лоту (якщо немає ставок)
- `/close_lot/<int:lot_id>`: Ручне закриття лоту
- `/start_auction/<int:lot_id>`: Публікація лоту для аукціону
- `/completed_lots`: Архів завершених лотів з пошуком

4. Бізнес-логіка

- Перевірка прав доступу (чи є користувач власником лоту)
- Валідація ставок (чи вища за поточну ціну, чи не власник робить ставку)
- Автоматичне оновлення ціни лоту при новій ставці
- Фонова перевірка переможця в окремому потоці
- Обробка завантаження зображень для лотів

5. Шаблони (Templates)

- Використовуються Jinja2 шаблони для відображення HTML сторінок
- Передаються списки лотів, інформація про конкретний лот, повідомлення про помилки/успіх

6. Особливості реалізації

- Використання сесій для авторизації
- Асинхронна перевірка переможця через Thread
- Фільтрація даних у БД за різними критеріями (активні/завершені лоти)

- Захист від несанкціонованого доступу (перевірка owner_id)
- Flash-повідомлення для інформування користувача

7. Запуск додатку

- Додаток запускається у режимі debug (для розробки)
- Автоматичне створення БД та папки для завантажень при першому запуску

Система реалізує повний цикл аукціону від створення лоту до визначення переможця, з обов'язковою авторизацією та перевіркою прав доступу на всіх етапах.

Command.py:

Клас DatabaseInitializer (неявний, реалізований через SQLAlchemy)

Відповідає за первинну ініціалізацію структури бази даних.

Характеристики:

- Використовує екземпляр Flask додатка (app)
- Працює з об'єктом SQLAlchemy (db)

Методи:

1. create_all()

- *Призначення:* Створення всіх таблиць у базі даних
- *Параметри:* Не потребує явних параметрів
- *Логіка роботи:*
 1. Аналізує всі визначені моделі (класи, що успадковують від db.Model)
 2. Генерує відповідні SQL-запити CREATE TABLE
 3. Виконує запити до підключеної бази даних
- *Обмеження:*
 - Створює лише нові таблиці (не оновлює існуючі)
 - Не підтримує міграції

Контекстні характеристики:

- Вимагає активного Flask application context (app.app_context())
- Використовує конфігурацію з app.config['SQLALCHEMY_DATABASE_URI']
- Автоматично визначає тип СУБД (SQLite/PostgreSQL/MySQL тощо)

Створювані сутності:

- Таблиця user (відповідає моделі User)
- Таблиця lot (відповідає моделі Lot)
- Таблиця bid (відповідає моделі Bid)
- Всі необхідні індекси та обмеження (foreign keys, unique constraints)

Типові сценарії використання:

1. Перший запуск додатку
2. Додавання нових моделей
3. Тестування (з нульовою БД)

Залежності:

- Вимагає попереднього визначення моделей (User, Lot, Bid)
- Залежить від коректної конфігурації SQLAlchemy
- Працює лише при активному Flask-додатку

Безпека:

- Ідемпотентна операція (багаторазовий виклик безпечний)
- Не впливає на існуючі дані у таблицях
- Не виконує операцій DROP TABLE

Цей код є критично важливим для початкової настройки системи, але не використовується під час регулярної роботи додатка після першого запуску.

Моделі бази даних (database.py):

Клас **User** представляє користувачів системи:

- Має унікальні поля: id (первинний ключ), username, email
- Містить поле password для зберігання паролю
- Пов'язаний з моделями Lot та Bid через відношення

Клас **Lot** представляє аукціонні лоти:

- Містить основні поля: id, name, description
- Цінові поля: start_price, current_price
- Поля статусів: is_active, is_closed, is_published
- Часівники: created_at, closed_at, end_time

- Поле `image_path` для зображень лоту
- Зв'язки з `User` (власник) та `Bid` (ставки)

Клас **`Bid`** фіксує ставки:

- Поле `amount` - сума ставки
- Зовнішні ключі `user_id` та `lot_id`
- Час створення `created_at`
- Зв'язки з `User` та `Lot`

Налаштування додатку (`app.py`):

Функція **`local_time()`** конвертує UTC час у локальний (за замовчуванням Київ).
Доступна в шаблонах через **`inject_local_time()`**.

При запуску автоматично:

- Створюються таблиці БД (**`db.create_all()`**)
- Створюється папка для завантажень (якщо відсутня)

Основні функції:

`check_winner()` - визначає переможця аукціону:

- Запускається у фоновому потоці після кожної ставки
- Чекає 120 секунд
- Якщо нових ставок немає - закриває лот і фіксує переможця

Маршрути (`routes`):

`index()` - головна сторінка зі списком активних лотів:

- Підтримує пошук за назвою/описом

`lot()` - сторінка лоту:

- GET: відображає інформацію про лот і ставки
- POST: обробляє нові ставки з перевітками:
 - Авторизація користувача
 - Чи не є користувач власником
 - Чи вища ставка за поточну ціну

Авторизація:

- **`register()`** - реєстрація нового акаунта
- **`login()`** - вхід в систему
- **`logout()`** - вихід

Управління лотами:

- **create_lot()** - створення нового лоту
- **my_lots()** - перегляд власних лотів
- **close_lot()** - закриття аукціону
- **start_auction()** - публікація лоту
- **edit_lot()** - редагування (якщо немає ставок)
- **completed_lots()** - перегляд завершених аукціонів

Ініціалізація БД (command.py):

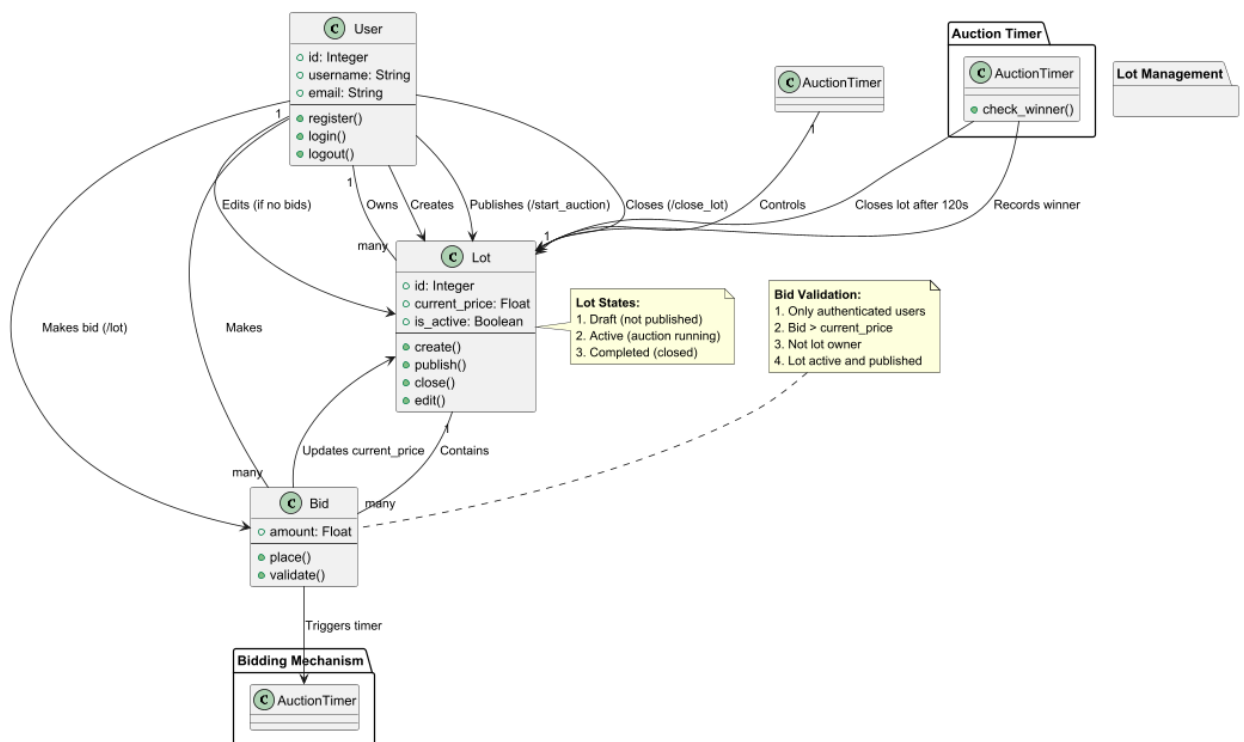
- **db.create_all()** - створює структуру БД при першому запуску

Бізнес-правила:

- Тільки авторизовані користувачі можуть робити ставки
- Ставка повинна бути вищою за поточну ціну
- Власник не може робити ставки на свої лоти
- Лоти проходять стани: чернетка, активний, завершений
- Переможець визначається через 2 хвилини після останньої ставки

4.3 Компоненти рівня бізнес-логіки

Основні сутності та інтерфейси рівня доступу до даних наведені на рис. 4.3



Опис бізнес-логіки:

- Створення нового лоту (create_lot):
 - Тільки авторизовані користувачі
 - Обов'язкові поля: назва, опис, стартова ціна
 - Можливість завантаження зображення
 - Статус "чернетка" при створенні
- Публікація лоту (start_auction):
 - Тільки власник лоту
 - Зміна статусу на "активний"
 - Встановлення часу завершення (+7 днів)
- Закриття лоту (close_lot):
 - Тільки власник лоту

- Зміна статусу на "завершений"
- Фіксація часу закриття

2. Механізм ставок

- Розміщення ставки (lot):
 - Перевірка авторизації
 - Заборона ставок для власника
 - Ставка повинна бути > поточної ціни
 - Оновлення поточної ціни
 - Запуск таймера визначення переможця (2 хв)
- Визначення переможця (check_winner):
 - Якщо нових ставок немає 2 хвилини
 - Останній ставник - переможець
 - Автоматичне закриття лоту

3. Правила валідації

- Для ставок:
 - Тільки для активних лотів
 - Мінімальне збільшення ціни
 - Заборона зміни ціни після першої ставки
- Для редагування лоту:
 - Тільки для чернеток
 - Тільки власник може редагувати
 - Блокування змін після публікації

4. Життєвий цикл лоту

1. Чернетка (is_published=False)
2. Активний (is_active=True)
3. Завершений (is_active=False)

5. Пошук та фільтрація

- По активним лотам:
 - За назвою/описом

- Тільки опубліковані
- По завершенні лотам:
 - Фільтр за статусом
 - Пошук за назвою/описом

6. Безпека

- Перевірка прав власника для всіх операцій
- Валідація даних форми
- Захист від несанкціонованого доступу

4.4 Компоненти рівня авторизації

Основні сутності та інтерфейси рівня авторизації наведені на рис. 4.4

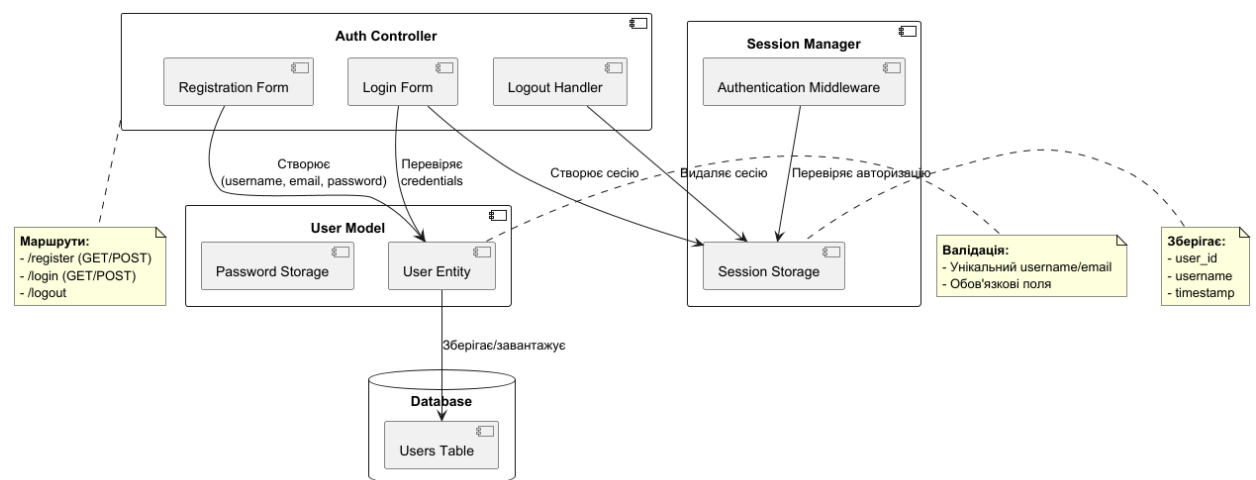


Рисунок 4.4 – Основні сутності рівня авторизації

Ключові компоненти системи авторизації:

1. **Auth Controller** - обробляє HTTP-запити:
 - Registration Form: форма реєстрації з валідацією
 - Login Form: форма входу з перевіркою облікових даних
 - Logout Handler: вихід з системи

2. **User Model** - відповідає за бізнес-логіку:

- User Entity: модель користувача з методами збереження/завантаження
- Password Storage: зберігання та перевірка паролів

3. **Session Manager** - керує сесіями:

- Session Storage: зберігає дані сесії
- Authentication Middleware: перевіряє авторизацію для захищених маршрутів

4. **Database** - постійне сховище даних:

- Users Table: таблиця користувачів у БД

Взаємодія між компонентами:

1. Реєстрація: Форма -> Створення User Entity -> Збереження в БД
2. Вхід: Форма -> Перевірка User Entity -> Створення сесії
3. Вихід: Видалення сесії
4. Перевірка прав доступу: Middleware -> Перевірка сесії

4.5 Компоненти рівня інтерфейсу користувача

Основні класи та інтерфейси рівня інтерфейсу користувача наведені на рис. 4.5

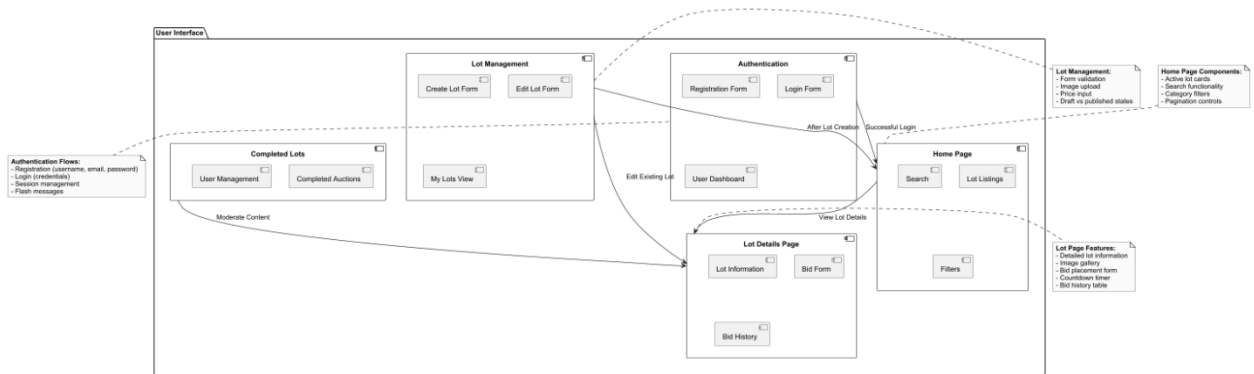


Рисунок 4.5 – Основні класи та інтерфейси рівня інтерфейсу користувача

Опис основних компонентів:

1. Головна сторінка

- Відображає активні лоти у вигляді карток
- Містить інструменти пошуку та фільтрації
- Пагінація для перегляду великої кількості лотів

2. Сторінка лоту

- Детальний опис товару/послуги
- Галерея зображень з можливістю перегляду
- Інтерактивна форма для розміщення ставок
- Відображення часу, що залишився до завершення
- Візуалізація історії ставок

3. Система автентифікації

- Форма реєстрації нового облікового запису
- Форма входу для існуючих користувачів
- Особистий кабінет з інформацією про користувача
- Система повідомлень про помилки/успішні дії

4. Інструменти керування

- Створення нового лоту з можливістю завантаження фото
- Редагування існуючих лотів (якщо немає ставок)
- Перегляд власних лотів у різних статусах

5. Адміністративний інтерфейс

- Перегляд завершених аукціонів
- Інструменти модерування контенту
- Керування користувацькими обліковками

Основні взаємодії:

- Всі форми мають валідацію в реальному часі
- Єдина система сповіщень про стан операцій
- Модальні вікна для важливих дій
- Адаптивний дизайн для різних пристроїв
- Обмеження доступу до функцій за ролями

Відповідність шаблонам:

- index.html - Головна сторінка
- lot.html - Сторінка лоту
- register.html - Реєстрація
- login.html - Вхід в систему
- create_lot.html - Створення лоту
- my_lots.html - Особистий кабінет
- edit_lot.html - Редагування
- completed_lots.html – Виконані лоти

Висновки:

Розробка цієї аукційної системи на Flask пройшла кілька ключових етапів, кожен з яких вніс важливий внесок у формування кінцевого продукту. Починаючи з проектування архітектури, ми ретельно продумали всі аспекти системи - від моделей даних до інтерфейсу користувача. Основу системи складає міцний фундамент у вигляді моделей User, Lot і Bid, які не лише визначають структуру даних, а й

містять важливу бізнес-логіку. Особливу увагу було приділено системі автентифікації, де реалізовано повний цикл роботи з користувачем: реєстрацію, вхід, вихід та контроль доступу.

Бізнес-логіка аукціону виявилась найбільш складною частиною проекту. Механізм ставок, з одного боку, має бути простим для користувачів, а з іншого - включає багато перевірок: чи авторизований користувач, чи не є він власником лоту, чи вища нова ставка за поточну ціну. Особливим викликом стала реалізація автоматичного визначення переможця через 2 хвилини після останньої ставки, що потребувало використання окремого потоку виконання.

Інтерфейсна частина системи була розроблена з урахуванням зручності користувача. Головна сторінка зі списком лотів, сторінка детального перегляду, форми створення та редагування лотів - кожен з цих елементів був продуманий так, щоб забезпечити інтуїтивно зрозумілий досвід взаємодії. Важливим аспектом стала реалізація пошуку та фільтрації, що дозволяє користувачам легко знаходити потрібні лоти.

Система управління доступом реалізує чітке розмежування прав: звичайні користувачі можуть робити ставки, власники лотів - керувати своїми аукціонами, а адміністратор має повний контроль над системою. Це досягається за рахунок перевірок у кожному маршруті та відповідних повідомлень про помилки.

Технічні аспекти, такі як робота з базою даних через SQLAlchemy, обробка файлів для завантаження зображень, робота з часовими поясами, були реалізовані з урахуванням найкращих практик. Особливу увагу приділено обробці помилок і наданню зрозумілих повідомлень користувачеві.

В результаті ми отримали цілісну систему, яка поєднує у собі складну бізнес-логіку аукціонів, зручний інтерфейс і надійну систему безпеки. Кожен компонент системи - від моделей даних до шаблонів - був ретельно продуманий і реалізований з урахуванням специфіки предметної області. Система готова до подальшого розширення, будь то додавання нових типів аукціонів, системи платежів або додаткових інструментів для адміністратора.

Проектування та розробка цієї системи продемонстрували важливість комплексного підходу, коли технічні рішення тісно пов'язані з бізнес-вимогами, а зручність користувача є пріоритетом на кожному етапі розробки. Отриманий досвід підтверджує, що навіть відносно невелика

система може містити багато нюансів, які необхідно враховувати для створення якісного продукту.

6 ПЕРЕЛІК ВИКОРИСТАНИХ ДЖЕРЕЛ

1. **Офіційна документація Flask**
<https://flask.palletsprojects.com/>
2. **Документація SQLAlchemy**
<https://www.sqlalchemy.org/>
3. **Документація PlantUML**
<https://plantuml.com/>
4. **Flask-SQLAlchemy документація**
<https://flask-sqlalchemy.palletsprojects.com/>
5. **WTForms документація (для форм)**
<https://wtforms.readthedocs.io/>
6. **Pytz документація (робота з часовими зонами)**
<https://pythonhosted.org/pytz/>
7. **Python threading**
<https://docs.python.org/3/library/threading.html>
8. **Рекомендації з безпеки Flask**
<https://flask.palletsprojects.com/en/2.3.x/security/>
9. **Шаблони проектування для веб-додатків**
https://developer.mozilla.org/en-US/docs/Web/Progressive_web_apps
10. **Принципи RESTful API**
<https://restfulapi.net/>

Усі конкретні рішення в цьому проекті були прийняті на основі комбінації загальних знань про веб-розробку та специфічних вимог до аукціонної системи.

7 ДОДАТКИ

Структура:

/auction_system/

- | /static/
- | | /uploads/
- | | style.css
- | /templates/
- | | base.html
- | | completed_lost.html
- | | create_lot.html
- | | edit_lot.html
- | | index.html
- | | login.html
- | | lot.html
- | | my_lots.html
- | | register.html
- | app.py
- | biznes.puml
- | command.py
- | database.py
- | diagram.puml
- | korist.puml
- | reg.puml
- | requirements.txt

Style.css:

```
body {
  font-family: Arial, sans-serif;
  margin: 20px;
  background-color: #f9f9f9;
}

h1, h2, h3 {
  color: #333;
}

.lots-container {
  display: flex;
  flex-wrap: wrap;
  gap: 20px;
```

```
        margin-top: 20px;
    }

    .lot-box {
        background-color: #fff;
        border: 1px solid #ddd;
        border-radius: 8px;
        padding: 15px;
        width: 250px;
        box-shadow: 0 2px 4px rgba(0, 0, 0, 0.1);
        text-align: center;
    }

    .lot-box h3 {
        margin: 10px 0;
        color: #333;
    }

    .lot-box p {
        margin: 5px 0;
        color: #555;
    }

    .lot-box .btn-open-lot {
        display: inline-block;
        margin-top: 10px;
        padding: 8px 16px;
        background-color: #007BFF;
        color: white;
        text-decoration: none;
        border-radius: 4px;
        transition: background-color 0.3s;
    }

    .lot-box .btn-open-lot:hover {
        background-color: #0056b3;
    }

    .lot-image {
        max-width: 100%;
        height: auto;
        border-radius: 8px;
    }

    .no-image {
        background-color: #eee;
        padding: 20px;
        border-radius: 8px;
        color: #777;
    }

    .btn-open-lot {
        display: inline-block;
        margin-top: 10px;
        padding: 8px 16px;
        background-color: #007BFF;
        color: white;
        text-decoration: none;
        border-radius: 4px;
        transition: background-color 0.3s;
    }

    .btn-open-lot:hover {
        background-color: #0056b3;
    }

    .btn-create-lot {
        display: inline-block;
```



```
margin-top: 20px;
padding: 10px 20px;
background-color: #28a745;
color: white;
text-decoration: none;
border-radius: 4px;
transition: background-color 0.3s;
}

.btn-create-lot:hover {
  background-color: #218838;
}

.lot-details {
  max-width: 600px;
  margin: 0 auto;
  background-color: #fff;
  padding: 20px;
  border-radius: 8px;
  box-shadow: 0 2px 4px rgba(0, 0, 0, 0.1);
}

.btn-back {
  display: inline-block;
  margin-top: 20px;
  padding: 8px 16px;
  background-color: #6c757d;
  color: white;
  text-decoration: none;
  border-radius: 4px;
  transition: background-color 0.3s;
}

.btn-back:hover {
  background-color: #5a6268;
}

.flash {
  padding: 10px;
  margin: 10px 0;
  border-radius: 4px;
}

.flash.success {
  background-color: #d4edda;
  color: #155724;
}

.flash.error {
  background-color: #f8d7da;
  color: #721c24;
}

nav {
  margin-bottom: 20px;
}

nav a {
  margin-right: 10px;
  text-decoration: none;
  color: #007bff;
}
```

```
nav a:hover {
  text-decoration: underline;
}

nav span {
  margin-left: 10px;
  color: #333;
}

.bid-form {
  margin-top: 20px;
}

.bid-form label {
  display: block;
  margin-bottom: 5px;
  font-weight: bold;
}

.bid-form input {
  width: 100%;
  padding: 8px;
  margin-bottom: 10px;
  border: 1px solid #ddd;
  border-radius: 4px;
}

.btn-bid {
  background-color: #28a745;
  color: white;
  border: none;
  padding: 8px 16px;
  border-radius: 4px;
  cursor: pointer;
}

.btn-bid:hover {
  background-color: #218838;
}

.bid-history {
  list-style-type: none;
  padding: 0;
}

.bid-history li {
  background-color: #f9f9f9;
  border: 1px solid #ddd;
  padding: 10px;
  margin-bottom: 5px;
  border-radius: 4px;
}

.completed-lots-container {
  display: flex;
  flex-wrap: wrap;
  gap: 20px;
}

.completed-lot-box {
```

```
background-color: #fff;
border: 1px solid #ddd;
border-radius: 8px;
padding: 15px;
width: 250px;
box-shadow: 0 2px 4px rgba(0, 0, 0, 0.1);
text-align: center;
}

.completed-lot-box h3 {
  color: #333;
}

.completed-lot-box p {
  margin: 5px 0;
}

.owner-actions {
  margin-top: 20px;
}

.btn-edit, .btn-close {
  display: inline-block;
  padding: 8px 16px;
  margin-right: 10px;
  color: white;
  text-decoration: none;
  border-radius: 4px;
  cursor: pointer;
}

.btn-edit {
  background-color: #ffc107;
}

.btn-edit:hover {
  background-color: #e0a800;
}

.btn-close {
  background-color: #dc3545;
}

.btn-close:hover {
  background-color: #c82333;
}

.owner-message {
  color: #dc3545;
  font-weight: bold;
  margin-top: 10px;
}

.auth-container {
  max-width: 400px;
  margin: 50px auto;
  padding: 20px;
  background-color: #fff;
  border-radius: 8px;
  box-shadow: 0 4px 6px rgba(0, 0, 0, 0.1);
}

.auth-container h2 {
  text-align: center;
}
```

```
    color: #333;
    margin-bottom: 20px;
}

.auth-form {
    display: flex;
    flex-direction: column;
    gap: 15px;
}

.form-group {
    display: flex;
    flex-direction: column;
}

.form-group label {
    margin-bottom: 5px;
    color: #555;
    font-weight: bold;
}

.form-group input,
.form-group textarea {
    width: 100%;
    padding: 10px;
    border: 1px solid #ddd;
    border-radius: 4px;
    font-size: 16px;
}

.form-group textarea {
    resize: vertical;
    min-height: 100px;
}

.form-group input[type="file"] {
    padding: 5px;
}

.btn-auth {
    background-color: #007BFF;
    color: white;
    border: none;
    padding: 10px;
    border-radius: 4px;
    font-size: 16px;
    cursor: pointer;
    transition: background-color 0.3s;
}

.btn-auth:hover {
    background-color: #0056b3;
}

.auth-link {
    text-align: center;
    margin-top: 15px;
    color: #555;
}

.auth-link a {
    color: #007BFF;
}
```

```
        text-decoration: none;
    }

    .auth-link a:hover {
        text-decoration: underline;
    }

    .search-form {
        margin-bottom: 20px;
        display: flex;
        gap: 10px;
    }

    .search-form input {
        flex: 1;
        padding: 10px;
        border: 1px solid #ddd;
        border-radius: 4px;
        font-size: 16px;
    }

    .search-form .btn-search {
        background-color: #007BFF;
        color: white;
        border: none;
        padding: 10px 20px;
        border-radius: 4px;
        cursor: pointer;
        transition: background-color 0.3s;
    }

    .search-form .btn-search:hover {
        background-color: #0056b3;
    }

    input:disabled {
        background-color: #f9f9f9;
        color: #777;
        cursor: not-allowed;
    }

    .navbar {
        display: flex;
        justify-content: space-between;
        align-items: center;
        background-color: #007BFF;
        padding: 10px 20px;
        box-shadow: 0 2px 4px rgba(0, 0, 0, 0.1);
    }

    .navbar-brand a {
        color: white;
        font-size: 24px;
        font-weight: bold;
        text-decoration: none;
    }

    .navbar-nav {
        display: flex;
        list-style: none;
        margin: 0;
        padding: 0;
        align-items: center;
    }

    .navbar-nav li {
```

```

        margin-left: 20px;
    }

    .navbar-nav li a {
        color: white;
        text-decoration: none;
        font-size: 16px;
        padding: 8px 12px;
        border-radius: 4px;
        transition: background-color 0.3s;
        display: flex;
        align-items: center;
        height: 100%;
    }

    .navbar-nav li a:hover {
        background-color: rgba(255, 255, 255, 0.1);
    }

    .navbar-nav .username {
        color: white;
        font-size: 16px;
        font-weight: bold;
        padding: 8px 12px;
        display: flex;
        align-items: center;
    }

    .btn-start-auction {
        background-color: #28a745;
        color: white;
        border: none;
        padding: 8px 16px;
        border-radius: 4px;
        cursor: pointer;
        transition: background-color 0.3s;
    }

    .btn-start-auction:hover {
        background-color: #218838;
    }

```

base.html:

```

<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Auction System</title>
    <link rel="stylesheet" href="{{ url_for('static', filename='styles.css') }}">
</head>
<body>
    <header>
        <nav class="navbar">
            <div class="navbar-brand">
                <a href="{{ url_for('index') }}">Auction System</a>
            </div>
            <ul class="navbar-nav">
                <li><a href="{{ url_for('index') }}">Home</a></li>
                <li><a href="{{ url_for('completed_lots') }}">Completed

```

```

Lots</a></li>
        {% if 'user_id' in session %}
            <li><a href="{{ url_for('my_lots') }}">My Lots</a></li>
            <li><a href="{{ url_for('create_lot') }}">Create
Lot</a></li>
            <li><a href="{{ url_for('logout') }}">Logout</a></li>
            <li class="username">{{ session['username'] }}</li>
        {% else %}
            <li><a href="{{ url_for('login') }}">Login</a></li>
            <li><a href="{{ url_for('register') }}">Register</a></li>
        {% endif %}
    </ul>
</nav>
</header>
<main>
    {% with messages = get_flashed_messages(with_categories=true) %}
        {% if messages %}
            {% for category, message in messages %}
                <div class="flash {{ category }}">{{ message }}</div>
            {% endfor %}
        {% endif %}
    {% endwith %}
    {% block content %}{% endblock %}
</main>
</body>
</html>

```

Completed_lots.html:

```

<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Auction System</title>
    <link rel="stylesheet" href="{{ url_for('static', filename='styles.css') }}">
</head>
<body>
    <header>
        <nav class="navbar">
            <div class="navbar-brand">
                <a href="{{ url_for('index') }}">Auction System</a>
            </div>
            <ul class="navbar-nav">
                <li><a href="{{ url_for('index') }}">Home</a></li>
                <li><a href="{{ url_for('completed_lots') }}">Completed
Lots</a></li>
                {% if 'user_id' in session %}
                    <li><a href="{{ url_for('my_lots') }}">My Lots</a></li>
                    <li><a href="{{ url_for('create_lot') }}">Create
Lot</a></li>
                    <li><a href="{{ url_for('logout') }}">Logout</a></li>
                    <li class="username">{{ session['username'] }}</li>
                {% else %}
                    <li><a href="{{ url_for('login') }}">Login</a></li>
                    <li><a href="{{ url_for('register') }}">Register</a></li>
                {% endif %}
            </ul>
        </nav>
    </header>

```

```

    <main>
    {% with messages = get_flashed_messages(with_categories=true) %}
    {% if messages %}
    {% for category, message in messages %}
    <div class="flash {{ category }}">{{ message }}</div>
    {% endfor %}
    {% endif %}
    {% endwith %}
    {% block content %}{% endblock %}
    </main>
</body>
</html>

```

Create_lot.html:

```

{% extends "base.html" %}

{% block content %}
    <div class="auth-container">
    <h2>Create New Lot</h2>
    <form method="POST" enctype="multipart/form-data">
    <div class="form-group">
    <label for="name">Name:</label>
    <input type="text" id="name" name="name" required>
    </div>
    <div class="form-group">
    <label for="description">Description:</label>
    <textarea id="description" name="description"
required></textarea>
    </div>
    <div class="form-group">
    <label for="start_price">Start Price:</label>
    <input type="number" id="start_price" name="start_price"
step="0.01" required>
    </div>
    <div class="form-group">
    <label for="image">Image:</label>
    <input type="file" id="image" name="image" accept="image/*">
    </div>
    <button type="submit" class="btn-auth">Create Lot</button>
    </form>
    <p class="auth-link"><a href="{{ url_for('index') }}">Back to
Home</a></p>
    </div>
{% endblock %}

```

Edit_lot.html:

```

{% extends "base.html" %}

{% block content %}
    <div class="auth-container">
    <h2>Edit Lot</h2>
    <form method="POST" class="auth-form">
    <div class="form-group">
    <label for="name">Name:</label>
    <input type="text" id="name" name="name" value="{{ lot.name }}" required>
    </div>

```



```

    <div class="form-group">
      <label for="description">Description:</label>
      <textarea id="description" name="description" required>{{ lot.description }}</textarea>
    </div>
    <div class="form-group">
      <label for="start_price">Start Price:</label>
      <input type="number" id="start_price" name="start_price" step="0.01" value="{{ lot.start_price }}" {% if
lot.bids %}disabled{% endif %} required>
    </div>
    <button type="submit" class="btn-auth">Save Changes</button>
  </form>
  <p class="auth-link"><a href="{{ url_for('lot', lot_id=lot.id) }}">Back to Lot</a></p>
</div>
{% endblock %}

```

Index.html:

```

{% extends "base.html" %}

{% block content %}
  <h2>Active Lots</h2>
  <form method="GET" action="{{ url_for('index') }}" class="search-form">
    <input type="text" name="search" placeholder="Search by name or description" value="{{
request.args.get('search', '') }}">
    <button type="submit" class="btn-search">Search</button>
  </form>
  <div class="lots-container">
    {% for lot in lots %}
      <div class="lot-box">
        {% if lot.image_path %}
          
        {% else %}
          <div class="no-image">No Image</div>
        {% endif %}
        <h3>{{ lot.name }}</h3>
        <p><strong>Current Price:</strong> ${{ lot.current_price }}</p>
        <p><strong>Created At:</strong> {{ lot.created_at.strftime('%Y-%m-%d %H:%M:%S') }}</p>
        <p><strong>Owner:</strong> {{ lot.owner.username }}</p>
        <a href="{{ url_for('lot', lot_id=lot.id) }}" class="btn-open-lot">Open Lot</a>
      </div>
    {% endfor %}
  </div>
  {% if 'user_id' in session %}
    <a href="{{ url_for('create_lot') }}" class="btn-create-lot">Create New Lot</a>
  {% else %}
    <p><a href="{{ url_for('login') }}">Log in</a> to create a lot.</p>
  {% endif %}
{% endblock %}

```

Login.html:

```

{% extends "base.html" %}

{% block content %}
  <div class="auth-container">

```

```

<h2>Login</h2>
<form method="POST" class="auth-form">
  <div class="form-group">
    <label for="username">Username:</label>
    <input type="text" id="username" name="username" required>
  </div>
  <div class="form-group">
    <label for="password">Password:</label>
    <input type="password" id="password" name="password" required>
  </div>
  <button type="submit" class="btn-auth">Log in</button>
</form>
<p class="auth-link">Don't have an account? <a href="{{ url_for('register') }}">Register</a></p>
</div>
{% endblock %}

```

Lot.html:

```

{% extends "base.html" %}

{% block content %}
  <div class="lot-details">
    <h2>{{ lot.name }}</h2>
    {% if lot.image_path %}
      
    {% else %}
      <div class="no-image">No Image</div>
    {% endif %}
    <p><strong>Description:</strong> {{ lot.description }}</p>
    <p><strong>Current Price:</strong> ${{ lot.current_price }}</p>
    <p><strong>Created At:</strong> {{ local_time(lot.created_at).strftime('%Y-%m-%d %H:%M:%S') }}</p>
    <p><strong>End Time:</strong> {{ local_time(lot.end_time).strftime('%Y-%m-%d %H:%M:%S') }}</p>
    <p><strong>Owner:</strong> {{ lot.owner.username }}</p>
    {% if 'user_id' in session and session['user_id'] == lot.owner_id %}
      <div class="owner-actions">
        <a href="{{ url_for('edit_lot', lot_id=lot.id) }}" class="btn-edit">Edit Lot</a>
        <a href="{{ url_for('close_lot', lot_id=lot.id) }}" class="btn-close">Close Lot</a>
      </div>
      <p class="owner-message">You cannot place a bid on your own lot.</p>
    {% endif %}

    {% if 'user_id' in session and session['user_id'] != lot.owner_id %}
      <form method="POST" class="bid-form">
        <label for="bid_amount">Your Bid:</label>
        <input type="number" id="bid_amount" name="bid_amount" step="0.01" min="{{ lot.current_price + 0.01 }}" required>
        <button type="submit" class="btn-bid">Place Bid</button>
      </form>
      {% elif 'user_id' not in session %}
        <p><a href="{{ url_for('login') }}">Log in</a> to place a bid.</p>
      {% endif %}

    <h3>Bid History</h3>
    <ul class="bid-history">
      {% for bid in lot.bids %}
        <li>
          ${{ bid.amount }} by {{ bid.user.username }} at {{ bid.created_at.strftime('%Y-%m-%d %H:%M:%S') }}
        </li>
      {% endfor %}
    </ul>
  </div>
{% endblock %}

```

```

        {% endfor %}
    </ul>

    <a href="{{ url_for('index') }}" class="btn-back">Back to Home</a>
</div>
{% endblock %}

```

My_lots.html:

```

{% extends "base.html" %}

{% block content %}
    <h2>My Lots</h2>
    <div class="lots-container">
        {% for lot in lots %}
            <div class="lot-box">
                {% if lot.image_path %}
                    
                {% else %}
                    <div class="no-image">No Image</div>
                {% endif %}
                <h3>{{ lot.name }}</h3>
                <p><strong>Current Price:</strong> ${{ lot.current_price }}</p>
                <p><strong>Created At:</strong> {{ local_time(lot.created_at).strftime('%Y-%m-%d %H:%M:%S') }}</p>
                <p><strong>Status:</strong>
                    {% if lot.is_published %}
                        Published
                    {% else %}
                        Draft
                    {% endif %}
                </p>
                <a href="{{ url_for('lot', lot_id=lot.id) }}" class="btn-open-lot">Open Lot</a>
                {% if not lot.is_published %}
                    <form method="POST" action="{{ url_for('start_auction', lot_id=lot.id) }}" style="margin-top: 10px;">
                        <button type="submit" class="btn-start-auction">Start Auction</button>
                    </form>
                {% endif %}
            </div>
        {% endfor %}
    </div>
    <a href="{{ url_for('index') }}" class="btn-back">Back to Home</a>
{% endblock %}

```

Register.html:

```

{% extends "base.html" %}

{% block content %}
    <div class="auth-container">
        <h2>Register</h2>
        <form method="POST" class="auth-form">
            <div class="form-group">
                <label for="username">Username:</label>

```

```

        <input type="text" id="username" name="username" required>
    </div>
    <div class="form-group">
        <label for="email">Email:</label>
        <input type="email" id="email" name="email" required>
    </div>
    <div class="form-group">
        <label for="password">Password:</label>
        <input type="password" id="password" name="password" required>
    </div>
    <button type="submit" class="btn-auth">Register</button>
</form>
<p class="auth-link">Already have an account? <a href="{{ url_for('login') }}">Log in</a></p>
</div>
{% endblock %}

```

App.py:

```

from flask import Flask, render_template, request, redirect, url_for, flash, session
from datetime import datetime, timedelta
from database import db, User, Lot, Bid
import os
from threading import Thread
import time
import pytz

app = Flask(__name__)
app.config['SQLALCHEMY_DATABASE_URI'] = 'sqlite:///auction.db'
app.config['SQLALCHEMY_TRACK_MODIFICATIONS'] = False
app.config['UPLOAD_FOLDER'] = 'static/uploads'
app.secret_key = 'supersecretkey'

db.init_app(app)

def local_time(utc_time, timezone='Europe/Kiev'):
    utc_time = utc_time.replace(tzinfo=pytz.UTC)
    local_tz = pytz.timezone(timezone)
    return utc_time.astimezone(local_tz)

@app.context_processor
def inject_local_time():
    return dict(local_time=local_time)

with app.app_context():
    db.create_all()
    if not os.path.exists(app.config['UPLOAD_FOLDER']):
        os.makedirs(app.config['UPLOAD_FOLDER'])

def check_winner(lot_id):
    time.sleep(120)
    with app.app_context():
        lot = Lot.query.get(lot_id)
        if lot and lot.is_active:
            last_bid = Bid.query.filter_by(lot_id=lot_id).order_by(Bid.created_at.desc()).first()
            if last_bid:
                lot.is_active = False

```

```

        lot.closed_at = datetime.now()
        db.session.commit()
        print(f"Lot {lot_id} won by user {last_bid.user_id}")

@app.route('/')
def index():
    search_query = request.args.get('search', "").strip()
    if search_query:
        lots = Lot.query.filter(
            (Lot.name.ilike(f'%{search_query}%')) | (Lot.description.ilike(f'%{search_query}%')),
            Lot.is_active == True,
            Lot.is_published == True
        ).all()
    else:
        lots = Lot.query.filter_by(is_active=True, is_published=True).all()
    return render_template('index.html', lots=lots)
@app.route('/lot/<int:lot_id>', methods=['GET', 'POST'])
def lot(lot_id):
    lot = Lot.query.get_or_404(lot_id)
    if request.method == 'POST':
        if 'user_id' not in session:
            flash('You need to log in to place a bid!', 'error')
            return redirect(url_for('login'))

        if session['user_id'] == lot.owner_id:
            flash('You cannot place a bid on your own lot!', 'error')
            return redirect(url_for('lot', lot_id=lot_id))

        bid_amount = float(request.form['bid_amount'])
        if bid_amount <= lot.current_price:
            flash('Your bid must be higher than the current price!', 'error')
            return redirect(url_for('lot', lot_id=lot_id))

        new_bid = Bid(
            amount=bid_amount,
            user_id=session['user_id'],
            lot_id=lot_id
        )
        lot.current_price = bid_amount
        db.session.add(new_bid)
        db.session.commit()

        Thread(target=check_winner, args=(lot_id,)).start()

        flash('Your bid has been placed!', 'success')
        return redirect(url_for('lot', lot_id=lot_id))

    bids = Bid.query.filter_by(lot_id=lot_id).order_by(Bid.created_at.desc()).all()
    return render_template('lot.html', lot=lot, bids=bids)

@app.route('/register', methods=['GET', 'POST'])
def register():
    if request.method == 'POST':
        username = request.form['username']
        email = request.form['email']
        password = request.form['password']

```

```

    if User.query.filter_by(username=username).first():
        flash('Username already exists!', 'error')
        return redirect(url_for('register'))

    new_user = User(username=username, email=email, password=password)
    db.session.add(new_user)
    db.session.commit()
    flash('Registration successful! Please log in.', 'success')
    return redirect(url_for('login'))
return render_template('register.html')

@app.route('/login', methods=['GET', 'POST'])
def login():
    if request.method == 'POST':
        username = request.form['username']
        password = request.form['password']

        user = User.query.filter_by(username=username, password=password).first()
        if user:
            session['user_id'] = user.id
            session['username'] = user.username
            flash('Login successful!', 'success')
            return redirect(url_for('index'))
        else:
            flash('Invalid username or password!', 'error')
    return render_template('login.html')

@app.route('/logout')
def logout():
    session.pop('user_id', None)
    session.pop('username', None)
    flash('You have been logged out.', 'success')
    return redirect(url_for('index'))

@app.route('/create_lot', methods=['GET', 'POST'])
def create_lot():
    if 'user_id' not in session:
        flash('You need to log in to create a lot!', 'error')
        return redirect(url_for('login'))

    if request.method == 'POST':
        name = request.form['name']
        description = request.form['description']
        start_price = float(request.form['start_price'])
        owner_id = session['user_id']

        image = request.files['image']
        if image:
            filename = os.path.join(app.config['UPLOAD_FOLDER'], image.filename)
            image.save(filename)
            image_path = f"uploads/{image.filename}"
        else:
            image_path = None

    new_lot = Lot(

```

```

        name=name,
        description=description,
        start_price=start_price,
        current_price=start_price,
        owner_id=owner_id,
        is_published=False,
        end_time=datetime.now() + timedelta(days=7),
        image_path=image_path
    )
    db.session.add(new_lot)
    db.session.commit()
    flash('Lot created successfully! Start the auction to make it visible.', 'success')
    return redirect(url_for('my_lots'))

    return render_template('create_lot.html')
@app.route('/my_lots')
def my_lots():
    if 'user_id' not in session:
        flash('You need to log in to view your lots!', 'error')
        return redirect(url_for('login'))

    user_lots = Lot.query.filter_by(owner_id=session['user_id']).all()
    return render_template('my_lots.html', lots=user_lots)

@app.route('/close_lot/<int:lot_id>')
def close_lot(lot_id):
    if 'user_id' not in session:
        flash('You need to log in to close a lot!', 'error')
        return redirect(url_for('login'))

    lot = Lot.query.get_or_404(lot_id)
    if lot.owner_id != session['user_id']:
        flash('You are not the owner of this lot!', 'error')
        return redirect(url_for('lot', lot_id=lot_id))

    lot.is_active = False
    lot.is_closed = True
    lot.closed_at = datetime.now()
    db.session.commit()
    flash('Lot closed successfully!', 'success')
    return redirect(url_for('lot', lot_id=lot_id))

@app.route('/start_auction/<int:lot_id>', methods=['POST'])
def start_auction(lot_id):
    if 'user_id' not in session:
        flash('You need to log in to start an auction!', 'error')
        return redirect(url_for('login'))

    lot = Lot.query.get_or_404(lot_id)
    if lot.owner_id != session['user_id']:
        flash('You are not the owner of this lot!', 'error')
        return redirect(url_for('my_lots'))

    lot.is_published = True
    db.session.commit()
    flash('Auction started successfully! The lot is now visible to all users.', 'success')
    return redirect(url_for('my_lots'))

```

```

@app.route('/edit_lot/<int:lot_id>', methods=['GET', 'POST'])
def edit_lot(lot_id):
    if 'user_id' not in session:
        flash('You need to log in to edit a lot!', 'error')
        return redirect(url_for('login'))

    lot = Lot.query.get_or_404(lot_id)
    if lot.owner_id != session['user_id']:
        flash('You are not the owner of this lot!', 'error')
        return redirect(url_for('lot', lot_id=lot_id))

    if request.method == 'POST':
        lot.name = request.form['name']
        lot.description = request.form['description']

        if not lot.bids:
            lot.start_price = float(request.form['start_price'])
            lot.current_price = float(request.form['start_price'])

        db.session.commit()
        flash('Lot updated successfully!', 'success')
        return redirect(url_for('lot', lot_id=lot_id))

    return render_template('edit_lot.html', lot=lot)

@app.route('/completed_lots')
def completed_lots():
    search_query = request.args.get('search', '').strip()
    if search_query:
        completed_lots = Lot.query.filter(
            (Lot.name.ilike(f'%{search_query}%')) | (Lot.description.ilike(f'%{search_query}%')),
            (Lot.is_active == False) | (Lot.is_closed == True),
            Lot.is_published == True
        ).all()
    else:
        completed_lots = Lot.query.filter(
            (Lot.is_active == False) | (Lot.is_closed == True),
            Lot.is_published == True
        ).all()
    return render_template('completed_lots.html', completed_lots=completed_lots)

if __name__ == '__main__':
    app.run(debug=True)

```

biznes.puml:

```

@startuml AuctionSystem
class User {
+ id: Integer
+ username: String
+ email: String
--
+ register()
+ login()

```



```

+ logout()
}

class Lot {
+ id: Integer
+ current_price: Float
+ is_active: Boolean
--
+ create()
+ publish()
+ close()
+ edit()
}

class Bid {
+ amount: Float
--
+ place()
+ validate()
}

package "Lot Management" {
User --> Lot : "Creates"
User --> Lot : "Publishes (/start_auction)"
User --> Lot : "Closes (/close_lot)"
User --> Lot : "Edits (if no bids)"
}

package "Bidding Mechanism" {
User --> Bid : "Makes bid (/lot)"
Bid --> Lot : "Updates current_price"
Bid --> AuctionTimer : "Triggers timer"
}

package "Auction Timer" {
class AuctionTimer {
+ check_winner()
}
AuctionTimer --> Lot : "Closes lot after 120s"
AuctionTimer --> Lot : "Records winner"
}

note top of Bid
**Bid Validation:**
1. Only authenticated users
2. Bid > current_price
3. Not lot owner
4. Lot active and published
end note

note right of Lot
**Lot States:**
1. Draft (not published)
2. Active (auction running)
3. Completed (closed)
end note

```

```
User "1" -- "many" Lot : "Owns"
User "1" -- "many" Bid : "Makes"
Lot "1" -- "many" Bid : "Contains"
AuctionTimer "1" -- "1" Lot : "Controls"
```

@enduml

Command.py:

```
from app import app, db
with app.app_context():
    db.create_all()
```

database.py:

```
from flask_sqlalchemy import SQLAlchemy

db = SQLAlchemy()

class User(db.Model):
    id = db.Column(db.Integer, primary_key=True)
    username = db.Column(db.String(80), unique=True, nullable=False)
    email = db.Column(db.String(120), unique=True, nullable=False)
    password = db.Column(db.String(120), nullable=False)

class Lot(db.Model):
    id = db.Column(db.Integer, primary_key=True)
    name = db.Column(db.String(100), nullable=False)
    description = db.Column(db.Text, nullable=False)
    start_price = db.Column(db.Float, nullable=False)
    current_price = db.Column(db.Float, nullable=False)
    owner_id = db.Column(db.Integer, db.ForeignKey('user.id'), nullable=False)
    is_active = db.Column(db.Boolean, default=True)
    is_closed = db.Column(db.Boolean, default=False)
    is_published = db.Column(db.Boolean, default=False)
    created_at = db.Column(db.DateTime, default=db.func.current_timestamp())
    closed_at = db.Column(db.DateTime)
    end_time = db.Column(db.DateTime)
    image_path = db.Column(db.String(200))

    owner = db.relationship('User', backref=db.backref('lots', lazy=True))

class Bid(db.Model):
    id = db.Column(db.Integer, primary_key=True)
    amount = db.Column(db.Float, nullable=False)
    user_id = db.Column(db.Integer, db.ForeignKey('user.id'), nullable=False)
    lot_id = db.Column(db.Integer, db.ForeignKey('lot.id'), nullable=False)
    created_at = db.Column(db.DateTime, default=db.func.current_timestamp())

    user = db.relationship('User', backref=db.backref('bids', lazy=True))
    lot = db.relationship('Lot', backref=db.backref('bids', lazy=True))
```

diagram.puml:

@startuml AuctionSystem

```

class User {
+ id: Integer
+ username: String
+ email: String
+ password: String
+ lots: List<Lot>
+ bids: List<Bid>
}

class Lot {
+ id: Integer
+ name: String
+ description: Text
+ start_price: Float
+ current_price: Float
+ is_active: Boolean
+ is_closed: Boolean
+ is_published: Boolean
+ created_at: DateTime
+ closed_at: DateTime
+ end_time: DateTime
+ image_path: String
+ bids: List<Bid>
}

class Bid {
+ id: Integer
+ amount: Float
+ created_at: DateTime
+ user: User
+ lot: Lot
}

User "1" *-- "0..*" Lot : owns
User "1" *-- "0..*" Bid : places
Lot "1" *-- "0..*" Bid : has

class FlaskApp {
+ __init__()
+ config
+ db: SQLAlchemy
+ secret_key
}

class Routes {
+ / : index()
+ /lot/<int:lot_id> : lot()
+ /register : register()
+ /login : login()
+ /logout : logout()
+ /create_lot : create_lot()
+ /my_lots : my_lots()
+ /close_lot/<int:lot_id> : close_lot()
+ /start_auction/<int:lot_id> : start_auction()
+ /edit_lot/<int:lot_id> : edit_lot()
+ /completed_lots : completed_lots()
}

class Helpers {

```

```

+ local_time(utc_time, timezone): DateTime
+ check_winner(lot_id): void
+ inject_local_time(): dict
}

```

```

class Commands {
+ db.create_all()
}

```

```

FlaskApp --> User : manages
FlaskApp --> Lot : manages
FlaskApp --> Bid : manages
FlaskApp ..> Routes : uses
FlaskApp ..> Helpers : includes
Commands ..> FlaskApp : initializes

```

```

note top of FlaskApp
Конфигурация:
- SQLAlchemy_DATABASE_URI
- UPLOAD_FOLDER
- SECRET_KEY

```

```
end note
```

```

note right of Helpers
Вспомогательные функции:
- local_time: конвертация времени
- check_winner: проверка победителя
- inject_local_time: для шаблонов

```

```
end note
```

```
@enduml
```

Korist.puml:

```
@startuml UI_Components
```

```

skinparam monochrome true
skinparam shadowing false
skinparam defaultFontName Arial

```

```

package "User Interface" {
    component "Home Page" as home {
        [Lot Listings]
        [Search]
        [Filters]
    }

    component "Lot Details Page" as lot {
        [Lot Information]
        [Bid Form]
        [Bid History]
    }
}

```

```

component "Authentication" as auth {
    [Registration Form]
    [Login Form]
    [User Dashboard]
}

component "Lot Management" as management {
    [Create Lot Form]
    [Edit Lot Form]
    [My Lots View]
}

component "Completed Lots" as admin {
    [Completed Auctions]
    [User Management]
}
}

```

home --> lot : "View Lot Details"
auth --> home : "Successful Login"
management --> home : "After Lot Creation"
management --> lot : "Edit Existing Lot"
admin --> lot : "Moderate Content"

note top of home

- **Home Page Components:****
- Active lot cards
 - Search functionality
 - Category filters
 - Pagination controls

end note

note right of lot

- **Lot Page Features:****
- Detailed lot information
 - Image gallery
 - Bid placement form
 - Countdown timer
 - Bid history table

end note

note bottom of auth

- **Authentication Flows:****
- Registration (username, email, password)
 - Login (credentials)
 - Session management
 - Flash messages

end note

note left of management

- **Lot Management:****
- Form validation
 - Image upload
 - Price input
 - Draft vs published states

end note

@enduml

Reg.puml:

```
@startuml AuthComponents

component "Auth Controller" as AuthController {
    [Registration Form]
    [Login Form]
    [Logout Handler]
}

component "User Model" as UserModel {
    [User Entity]
    [Password Storage]
}

component "Session Manager" as Session {
    [Session Storage]
    [Authentication Middleware]
}

database "Database" as DB {
    [Users Table]
}

[Registration Form] --> [User Entity] : "Створює\n(username, email, password)"
[Login Form] --> [User Entity] : "Перевіряє\ncredentials"
[User Entity] --> [Users Table] : "Зберігає/завантажує"
[Login Form] --> [Session Storage] : "Створює сесію"
[Logout Handler] --> [Session Storage] : "Видаляє сесію"
[Authentication Middleware] --> [Session Storage] : "Перевіряє авторизацію"

note right of [User Entity]
    **Валідація:**
    - Унікальний username/email
    - Обов'язкові поля
end note

note left of [Session Storage]
    **Зберігає:**
    - user_id
    - username
    - timestamp
end note

note bottom of AuthController
    **Маршрути:**
    - /register (GET/POST)
    - /login (GET/POST)
    - /logout
end note

@enduml
```

Requirements.txt:

```
Flask==2.3.2  
Flask-SQLAlchemy==3.0.5
```